Amaterasu: Next Token Prediction Through Semantic Codewords

Átila Araujo Lobo[a,b*]

[a] Department of Electrical Engineering, University of Brasília, Brazil
[b] Technische Universität Graz, Austria

Abstract

This work proposes a language model with 2.1M parameters that achieves 22.08% top-1 and 29.92% top-10 accuracy on held-out next-token prediction within the same corpus distribution. Instead of predicting a probability distribution over the vocabulary, the model represents tokens as fixed binary codewords and predicts the probability of each bit independently. To encode contextual information, the model employs a nonlinear dynamical system to capture semantic information through the evolution of context matrices. Sequences of context matrices are projected into a lower-dimensional subspace to obtain a compact representation of long-range dependencies. The contribution of the context representation is evaluated through ablation experiments, showing a statistically significant decrease in prediction accuracy when the context matrices are perturbed.

Keywords: Language Modeling; Binary Code Prediction; Error-Correcting Output Codes; Dynamical Systems.

Introduction

Even relatively small language models such as GPT-2 small (117M) require a number of parameters that makes them impractical on devices without hardware acceleration. This motivates the development of alternative architectures that reduce parameter count while maintaining reasonable predictive performance over large vocabularies.

In this work, the architecture developed predicts bitwise probabilities of token codewords instead of tokens themselves. That is, instead of predicting a full probability distribution over a vocabulary of approximately 56k tokens, the model predicts a 48-bit codeword associated with each token. These codewords are constructed directly from vector embeddings spaCy [1], enabling each bit to retain semantic information by design. A probability distribution over tokens is then recovered through a scoring function based on the predicted bitwise probabilities.

To model sequential context, the proposed architecture replaces attention mechanisms with a nonlinear dynamical system that evolves a particular set of matrices called context matrices. These matrices encode information from arbitrarily long input sequences in a compressed form. The resulting representation is projected into a lower-dimensional context vector using principal component analysis (PCA), which serves as input to the prediction network.

The resulting model contains approximately 2 million parameters and achieves top-1 and top-10 accuracies of 22.08% and 29.92%, respectively, on held-out next-token prediction within the same corpus. These results demonstrate that nontrivial next-token prediction performance can be achieved with significantly fewer parameters by combining structured codewords with dynamical context representations.

* Corresponding author. Address: Department of Electrical Engineering, University of Brasília, Brazil. Email: loboatila@gmail.com

Related Work

In this work, the output of the the neural network is used to assign a score to each token's codeword of the vocabulary which is used to compute top-1 as well as top-10 most likely next tokens. Although similar to the idea of error correcting output codes (ECOC) [2] where codewords split classes based on individual bits, it differs since the token codewords are not optimized for error correction. Decoding is performed using a loss-based strategy, where candidate codewords are scored according to accuracy of individual bits obtained from training data, following the general framework proposed in [3]. In contrast to classical ECOC approaches, where each bit is predicted by an independent classifier, the present model produces all bit probabilities through a single neural network.

Oda et al. [4] pioneered binary code prediction for neural machine translation, mapping words to bit arrays via frequency rank. The work described in this article differs fundamentally in that binary codes are derived from PCA of token embeddings, so that each bit is endowed with semantic information.

The use of binary codewords whose individual bits reflect semantic structure is related to semantic hashing [5], where discrete representations are learned in order to preserve similarity relationships in embedding space. Another related work is [6], which also learns binary representations, but does so using an autoencoder-based architecture. In contrast, the present work derives codewords directly from the geometry of pretrained embeddings through deterministic partitioning, rather than learning them via optimization of an objective function.

The use of dynamical systems to retain past inputs and reduce computational cost has been explored in the context of reservoir computing. In [7], this approach improved autonomous driving accuracy by 12.7% to 42% and achieved a computational speedup of 1.71 to 2.2 times. Reservoir computing has also been applied to language models [8]. Although the present work also uses a dynamical system to encode past information, it differs from reservoir computing in several ways.

In reservoir computing, the update rule is:

$$x_{t+1} = f(Wx_t + Uu_t) \tag{1}$$

where x_t is the reservoir state, u_t the input signal, f a nonlinearity, and W and U random matrices whose spectral radius determines memory retention. In contrast, this work eliminates the input signal vector, using the update rule:

$$C_{t+1} = f(R_t C_t) \tag{2}$$

where R_t depends on the current token's vector embedding, and C_t is a context matrix which encodes information of the sentence meaning.

Additionally, the dynamical system in this work has an underlying geometric structure absent in reservoir computing. Specifically, the update rule normalizes the context matrix as the final step of the nonlinearity, bounding the matrix entries and ensuring stable dynamics over time. Finally, context matrices are reset at the start of each new sentence, which simplifies the learning problem of the network as it only needs to learn the system's dynamics near the identity matrix.

The choice of nonlinearity was also partially motivated by recent work suggesting that transformer representations may organize information in structured latent subspaces. In [9], selectively amplifying neurons associated with specific programming languages was shown to improve token prediction accuracy, while [10] provided evidence that semantic information in transformer sentence embeddings can be probed through approximately linear subspaces. In the present work, the nonlinear evolution of context matrices similarly causes certain rows to become

more prominent over time, effectively reducing the rank of the representation and emphasizing specific directions in the latent space.

In [11], techniques were proposed to reduce reservoir size by incorporating past reservoir states into the current output layer, including the use of delayed and drifting states. These approaches increase the effective dimensionality of the system by concatenating representations from multiple time steps.

The present work adopts a related approach of leveraging past states, however, it differs in several important aspects. Instead of concatenating raw states at fixed time steps, the proposed model constructs a sequence of context matrices summarizing entire phrases, which are then projected into a lower-dimensional representation before being processed by the neural network. This results in a compressed representation of longer temporal structure rather than an explicit expansion of dimensionality. Furthermore, the use of phrase-level segmentation introduces variable-length temporal units, in contrast to the fixed-step dynamics typically considered in reservoir computing.

In [12], a mechanism is introduced to extend context beyond a fixed length by caching past hidden states and reusing them during attention computation. This allows the model to access information from previous segments without recomputing representations.

In contrast, the present work does not reuse internal hidden states from a neural network. Instead, it constructs a separate dynamical system in which past information is encoded through the evolution of context matrices driven by token-dependent transformations. As a result, memory is not stored as intermediate activations, but rather accumulated through structured state transitions.

While the model described in this article uses rotation matrices to encode information related to the semantics of a sentence while not enforcing a positional representation explicitly, the work of [13] applies rotation matrices to query and key representations within the attention mechanism, enabling attention scores to depend on the relative positions between tokens.

Codeword and Rotation Matrices

Spacy large model [1] has an intrinsic dimension of 300 of its vector embeddings, but despite that, it may be contained in a much lower dimensional subspace. One can see that by analyzing the singular values obtained through PCA over the set of all vector embeddings as can be seen on Figure 1.

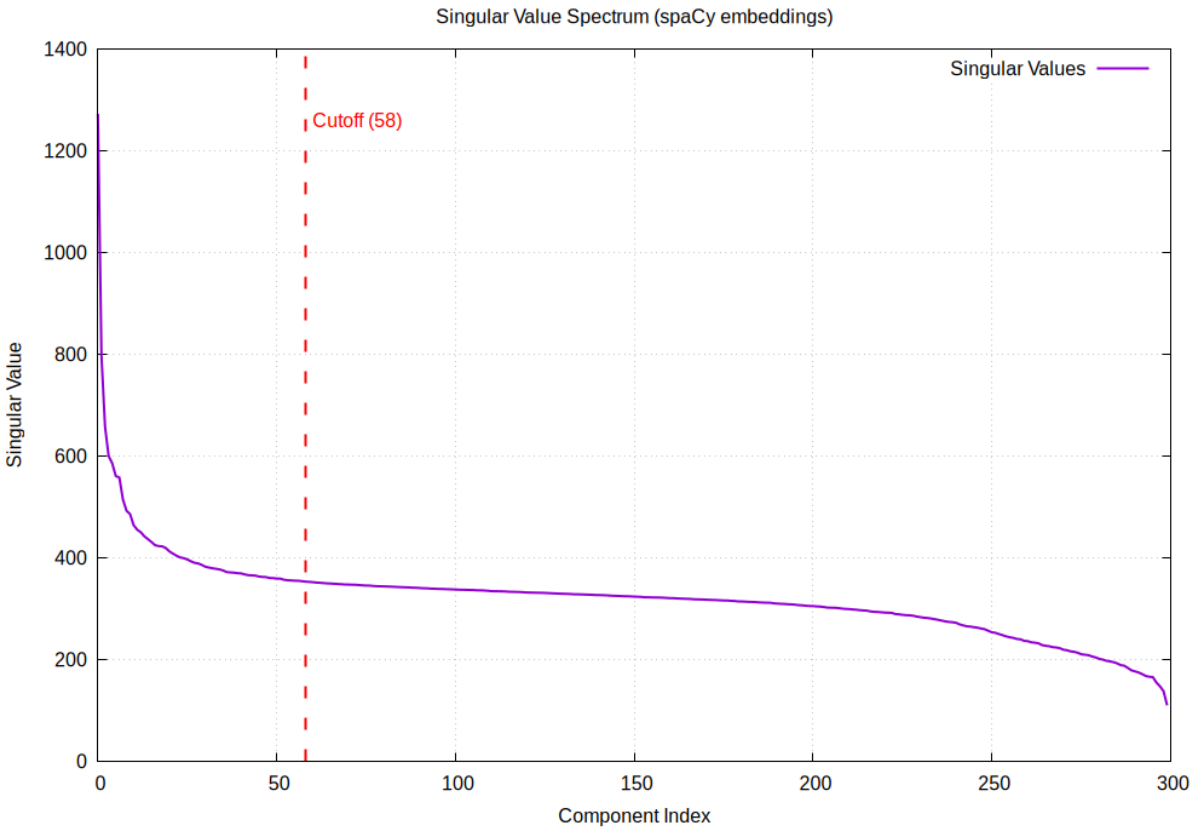

Figure 1. Singular value spectrum of the spaCy large-model vector embeddings. The embedding dimension was chosen based on the onset of a flattening region in the singular value spectrum, beginning approximately at component index 50.

As shown in [14], the first principal components of word embeddings are often associated with token frequency. Since Figure 1 indicates that most variation is concentrated within roughly the first 50 components, only principal components 3 to 58 were retained in order to reduce dimensionality while preserving most relevant information. This choice results in embeddings of dimension 55, whose significance will be discussed later.

It is commonly known that the order of words in a sentence influence the meaning of the sentence, therefore to generate a geometric representation that encodes context of a phrase, it is reasonable to use structures where the order of operation matters such as non-commutative Lie groups.

Following this line of thought, it was chosen to represent semantic information of an entire sentence in what was given the name of context matrix which is evolved by a non-linear dynamical system with the addition of each token of the sentence. In the update rule, the context matrix is multiplied by a rotation matrix of that corresponding token. I will now describe how those rotation matrices were generated in detail.

Let X be the canonical basis of skew-symmetric matrices in $\mathbb{R}^{11\times 11}$. Since $\dim(\mathrm{SO}(11)) = 55$, a one-to-one correspondence between the canonical basis of $\mathbb{R}^{55}$ and the elements of X induces a linear map

$$l : \mathbb{R}^{55} \to \langle X \rangle. \tag{3}$$

Because exponentiation maps skew-symmetric matrices to rotation matrices, each token embedding $v \in \mathbb{R}^{55}$ is mapped to a rotation matrix through

$$R(v) = \exp(\gamma l(v)), \tag{4}$$

where γ is a scaling parameter chosen so that the maximum rotation angle remains below approximately 0.05 radians ($2.86°$), preventing excessively erratic trajectories in the evolution of the context matrix.

The map l can only be bijective if the dimension of the embedding matches the dimension of $\mathrm{SO}(N)$ so that's why it was chosen a value of $N = 11$ so that the dimension of the vector embedding can be close to 50 or more specifically 55.

Context Matrix

As discussed in the Related Work section, the works of [9] and [10] provide evidence that transformer representations organize information in latent subspaces of hidden-layer activations. Although further studies are required to determine whether the context matrices proposed in this work similarly activate subspaces associated with particular semantic properties, the dynamical system was intentionally designed to reduce the effective rank of the representation, as this property may favor the emergence of structured semantic encodings.

To quantify the effective rank, we will use a metric that has formula closely related to the formula for entropy of a probability distribution and because of that it will be called the entropy of the context matrix.

Since we are defining a concept for entropy for matrices, we shall also define the energy of a row n of matrix C as

$$E_n(C) \coloneqq \sum_k C_{nk}^2 \tag{5}$$

The definition comes from the Frobenius norm so that it holds

$$\|C\|_{\mathrm{Fr}}^2 = \sum_n E_n(C) \tag{6}$$

From energy we can also define an "entropy" as

$$S(C) = -\sum_n p_n(C) \log(p_n(C)) \tag{7}$$

where each probability is given by

$$p_n(C) = \frac{E_n(C)}{\sum_i E_i(C)}. \tag{8}$$

The definition of entropy 7 is such that it decreases as most of the contribution to the Frobenius norm becomes dependent on certain rows. That's what was meant by effective rank where a low in entropy matrix means that most rows become negligible in magnitude when compared to certain rows of the matrix.

As stated prior an update rule for context matrices which is simply multiplication by a rotation matrix or

$$C_{n+1} = C_n R_n \tag{9}$$

cannot reduce rank since the energy of each row for a rotation matrix C is

$$E_n(C) = \sum_k C_{nk}^2 = \sum C_{nk} C_{kn}^T = \left(CC^T\right)_{nn} = 1 \tag{10}$$

and therefore the probability p_n becomes simply

$$p_n = \frac{1}{11} \tag{11}$$

which is fixed and therefore entropy cannot decrease with each iteration.

To make it entropy reducing, the update rule 9 was then replaced by the following

$$\begin{aligned} X &= C_n R_n \\ Y &= X + \alpha X A X^T X B \\ C_{n+1} &= \frac{Y}{\|Y\|} \end{aligned} \tag{12}$$

where the norm in the equation above being the normalized Frobenius norm being defined in this way

$$\|A\| = \frac{1}{\sqrt{D}} \|A\|_{\mathrm{Fr}} \tag{13}$$

The norm is such that for a rotation matrix R it holds

$$\|R\| = 1, \tag{14}$$

since

$$\left(R^T R\right)_{ij} = \delta_{ij}. \tag{15}$$

That choice of normalization implies that when $\alpha = 0$ the update rule 12 reduces to 9 since the context matrices would all be rotation matrices.

In attempt to increase the effective dimension of the context matrices, it was also tested separately in the model an update rule with stochastic parameters that could remove the systems from converging to attractors with the following variation of the update rule 12

$$\begin{aligned} X &= C_n R_n \\ Y &= X + \alpha X A X^T X B + \tilde{\varepsilon} \\ C_{n+1} &= \frac{Y}{\|Y\|} \end{aligned} \tag{16}$$

where $\tilde{\varepsilon}$ is a 11×11 matrix with entries sampled from a random uniorm distribution in the range $\left[-2 \times 10^{-2}, 2 \times 10^{-2}\right]$. What was referred to as the deterministic model uses update rule 12 while the stochastic model uses update rule 16.

It was chosen empirically α to be a random value in $[-0.05, 0.05]$, A to be a random matrix with values in range $[-0.5, 0.5]$ and B to be the sum of a diagonal matrix with entries in the range $[-0.5, 0.5]$ plus a random matrix with entries in the range $\left[-5 \times 10^{-3}, 5 \times 10^{-3}\right]$. All the distributions of the random matrices were uniform distributions and for the contribution of the nonlinearity to be dependent strictly on α, matrices A and B were divided by the normalized Frobenius norm after being randomly generated.

Phrases with different set of tokens lead to a variety of paths for the dynamical system each with their own distribution of energy on each row. For illustrative purposes, it was picked one arbitrary phrase and it was plotted energy and entropy of the context matrix in function of each token of the sentence. The results which are found in Figure 2 and Figure 3 show that most of the Frobenius

norm becomes concentrated in rows 6 and 7 as each iteration progresses and as consequence, the entropy of the context matrix is progressively reduced.

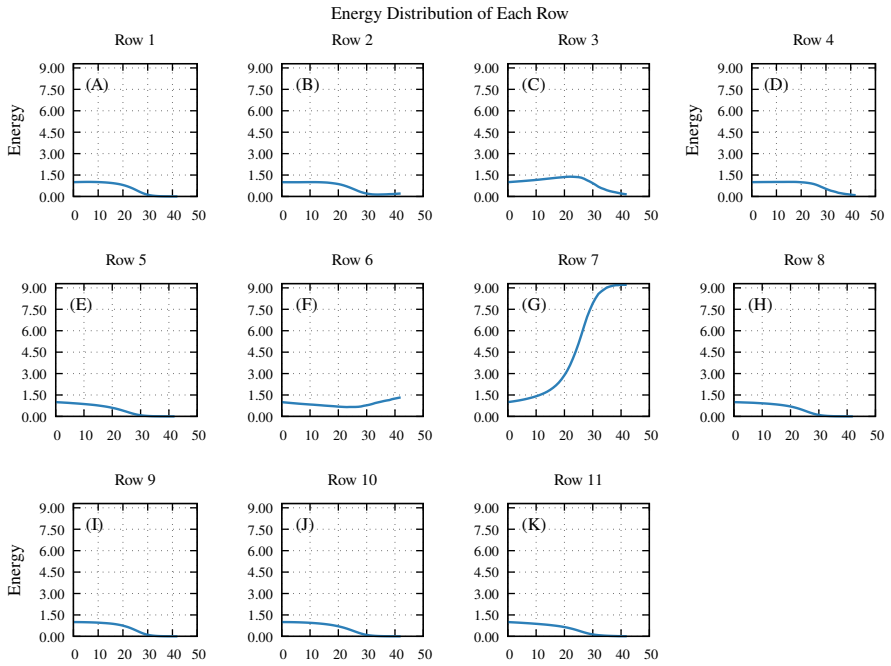

Figure 2. Plot of energy of each row of the context matrix in function of each tokens for the phrase: *"The electrical problems of the present day lie largely in the economical transmission of power and in the radical improvement of the means and methods of illumination."*

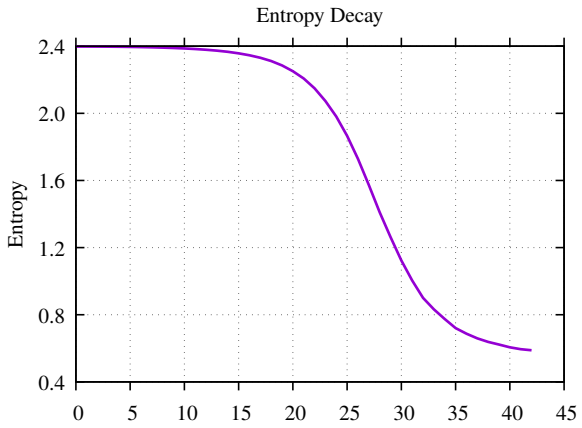

Figure 3. Plot of entropy of context matrix in function of each token for the phrase: *"The electrical problems of the present day lie largely in the economical transmission of power and in the radical improvement of the means and methods of illumination."*

Heuristic for Token Prediction

Although the architecture does not generate a probability distribution, it's possible to infer a probability distribution from the codeword of each token together with the probabilities of each individual bit outputted by the model. From this estimation of the PDF it is possible to derive top-1 and top-10 accuracy for next token prediction.

More specifically for each codeword of a token with individual bits being $\{b_i\}$, where the accuracy of each bit $\{\beta_i\}$ and the probability of each bit being b_i from the input vector x is given by the neural network by $P(b_i|x,i)$ then the score assigned to each codeword is

$$S(b) = \sum_i \alpha_i \log(P(b_i|x,i)) \tag{17}$$

with weight $\alpha_i = \beta_i - 0.5$ so that bits of greater accuracy have greater contribution while bits with 50% of accuracy have no contribution.

If the output at the i-th bit computed by the network is $p_i(x)$ then

$$P(1|x,i) = p_i(x) \tag{18}$$

and

$$P(0|x,i) = 1 - p_i(x) \tag{19}$$

therefore

$$P(b_i|x,i) = p_i^{b_i}(1-p_i)^{1-b_i} \tag{20}$$

which implies

$$\begin{aligned} S(b) = \sum_i \alpha_i (b_i \log(p_i) + (1-b_i)\log(1-p_i)) = \\ = \sum_i \alpha_i b_i (\log(p_i) - \log(1-p_i)) + \sum_i \alpha_i \log(1-p_i) = \\ = \sum_i \alpha_i b_i \ \text{logit}(p_i) + \sum_i \alpha_i \log(1-p_i) = \alpha \circ p \cdot \text{logit}\ (p) + \alpha \circ \log(1-p) \end{aligned} \tag{21}$$

The formula allows faster calculation of the score of each codeword by performing a dot product between vector of bits b and vector $\alpha \circ \ \text{logit}(p)$ with $\circ$ being the Hadamard product.

By exponentiation of the negative score followed by normalization, one can obtain a probability density estimation for each token. Since the token score was used merely to generate an order of most likely tokens, it was not computed during prediction the probability density, but simply the score instead.

To prevent data leakage, it was used only the training dataset to obtain the bit accuracy that would later be used to compute scores for each codeword.

Context Vectors

The token prediction relies on the context of the sentence and the meaning of past phrases. Therefore to encode the context, it was flattened a sequence of context matrices of the 10 past phrases together with the context matrix of the current phrase. This creates a vector of dimension $(10+1) \times 11 \times 11 = 1331$

It was performed PCA to search for the most significant principal components. To obtain a statistically likely value, it was generated flattened vectors of the context matrices in the way it was previously mentioned for each word of every sentence of 5 different books of the cleaned version of project Guttenberg. Then due to time constraints, it was shuffled randomly those flattened vectors and then selected the first 50000 samples to perform PCA. The singular values of the PCA are plotted in Figure 4.

From the PCA basis vectors it was constructed a projection matrix which was scaled by the inverse singular values of the corresponding components, yielding a variance-normalized (whitened) low-dimensional representation of the flattened context matrices.

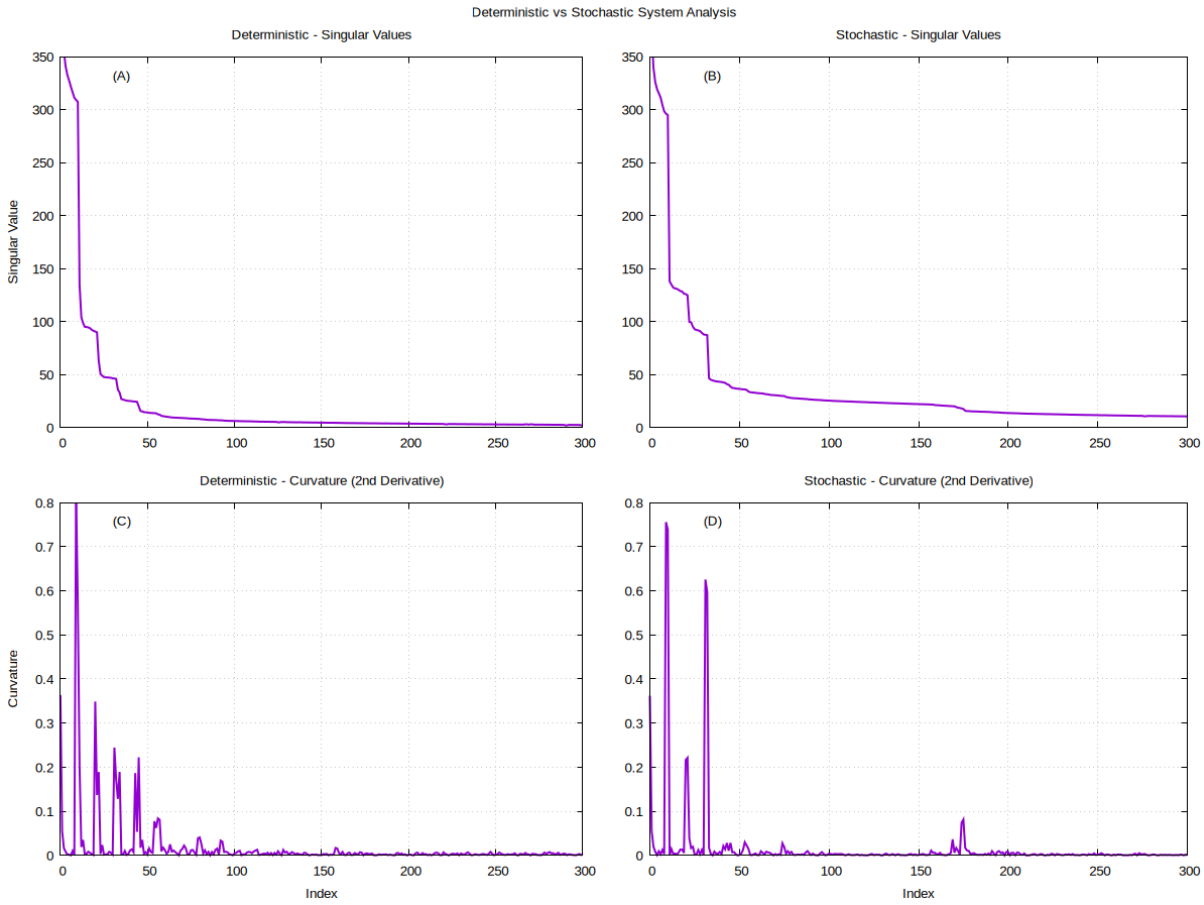

Figure 4. **Singular value spectra for the deterministic (A) and stochastic (B) models, and magnitude of the second-order finite difference of $\log(\sigma_i)$ for the deterministic (C) and stochastic (D) models.** The lower figures emphasize regions of changing curvature in the spectra.

The projection in the lower dimensional subspace of the flattened sequence of context matrices was called context vectors which would have 130 dimensions for the deterministic update rule and 170 dimensions for the stochastic update rule.

To obtain the projection matrix for the flattened sequence of context matrices, in a way that prevents data leakage, it was used a different set of books that were used to generate training, test and validation dataset.

Consider the following example of predictions made by the model

```
Input: Wilson stood in the position from which he had derived his nickname , his left
hand and left foot well to the front , his

Predictions:
hands - arms - sits - forehead - butt - mouth - locks - brow - finger
- kneels
```

As described in Section “Architecture”, the prediction model receives as input the concatenation of the context vector and the vector embeddings of the last four tokens. In this example, the last four tokens are 1. **the**, 2. **front**, 3. **“,”**, and 4. **his**, none of which are directly related to body parts.

Despite this, many of the predicted tokens (e.g. hands, arms, forehead, mouth, finger) are semantically associated with body parts, which appeared earlier in the sequence (e.g., hand, foot). While it is only a qualitative observation, it’s consistent with the hypothesis that information from

earlier parts of the sequence is retained in the context matrices and contributes to the prediction, beyond what is available from the local input window.

Another thing that is worth to be addressed briefly is that one can also see the formation of a stair-case pattern in the singular values on Figure 4. As can be seen on (B) and (D) of Figure 4, the addition of noise in the system does not seem to change the pattern.

The behavior is rather curious and perhaps each region of almost constant singular value correspond to an activated subspace loosely analogous to the subspaces of hidden layers that are activated by transformers when it's given input on a particular topic as it was previously mentioned. Further studies must be made to provide the necessary evidence to prove or falsify the hypothesis.

Dataset Generation

The corpus used to generate the dataset was provided by [15] which contained a list of books in text file format. Then it was selected a subset of books to generate a list of numpy integer arrays containing the index of each token in the sentence for every sentence, where sentences with less than 2 tokens were not included.

To generate the datasets, it was concatenated context vectors together with the vector embeddings (55 dimensions each) of the last 4 tokens for the input dataset. For the ouput dataset it was saved the 48 bit codeword of the next token as the task is next token prediction.

The dimension of the input dataset depended on the dimension of the subspace which the context matrices are projected on, where the dimension of each row of the input dataset for the deterministic rule was $130 + 4 \times 55 = 350$ and for the stochastic update rule the dimension of each row being $170 + 4 \times 55 = 390$.

The sequence of inputs and corresponding outputs were computed for 65 books, which were grouped into 5 at each iteration, forming in total 13 separate training datasets of approximately 8GB of data. After processing a group of 5 at each iteration, shuffling is performed so that the network can capture patterns of all the books of the group where at each iteration it is limited the number of samples in the training dataset to have at most 500,000 elements.

To prevent data leakage, it was separated a dataset of inputs and outputs with 10,000 elements that would not be used during training where half of those would be split into test dataset and the other half into validation dataset. The test/validation dataset would use the same books, as the training dataset but different parts of each sentence as well as different input/outputs.

Architecture

To make it simpler, in this section I will alternate between giving a short justification of each choice of component in the architecture followed by the actual tensorflow code. I will be describing the best performing model which was the deterministic model, but the architecture for the stochastic model is similar in all aspects with the only difference being the input dimension which would be 390 instead of 350. Have in mind that some of the layers have number of neurons equal to the input dimension and that as well is modified for the stochastic model.

```
inputs = tf.keras.Input(shape=(350,))
```

To increase training speed, the first layer multiplies each entry of the input by a learned scale so that the neural network learns to balance the contribution of each entry more quickly.

```
x = DimensionScaling(350)(inputs)
```

Then to model interactions between input features, the neural network starts with 2 layers, each with number of neurons equal to input dimension.

```
x = tf.keras.layers.Dense(350, activation='relu')(x)
x = tf.keras.layers.Dense(350, activation='relu')(x)
```

Then it passes the output by one dense layer that is then normalized and activated by GeLU with the choice of activation function being supported by the works of [16], [17]. Layer normalization was utilized so that the output is within the range where the action of GeLU is meaningful.

```
x = tf.keras.layers.Dense(350, activation=None)(x)
x = tf.keras.layers.LayerNormalization()(x)
x = tf.keras.layers.Activation('gelu')(x)
```

Then the output is mapped into a sequence of 256 vectors of features of 20 dimensions each.

```
x = tf.keras.layers.Dense(20<<8, activation='relu')(x) # 256 of dimension 20
```

The features are combined hierarchically in a binary tree structure across successive hidden layers. At each stage, pairs of feature vectors are concatenated and passed through a dense layer with 40 units and ReLU activation. The resulting representation is then mapped to a set of candidate features using a dense layer with 20 units. A gating mechanism, inspired by architectures such as LSTM [18] and RNN [19], is applied to selectively retain or suppress components of these candidate features. The gated output is subsequently passed through an additional ReLU-activated layer before proceeding to the next level of the hierarchy.

```
for N in range(7, 1, -1):
    x = tf.keras.layers.Reshape((1<<N, 2, 20))(x)
    x1 = x[:, :, 0, :]   # (batch, groups, 20)
    x2 = x[:, :, 1, :]   # (batch, groups, 20)

    concat = tf.keras.layers.Concatenate()([x1, x2])

    h = tf.keras.layers.Dense(40, activation='relu')(concat)

    gate = tf.keras.layers.Dense(20, activation='sigmoid')(h)
    candidate = tf.keras.layers.Dense(20)(h)

    merged = gate * candidate

    x = tf.keras.layers.Activation('relu')(merged)
```

To not map to a set of vectors with dimension lower than the output dimension, which may compromise the model, it was added a dense layer of 24 units so that the remaining two feature vectors when concatenated have a total of $2 \times 24 = 48$ dimensions.

```
x = tf.keras.layers.Reshape((2,40))(x)
x = tf.keras.layers.Dense(24, activation='relu')(x) # 2 of dimension 24 instead of 20
```

Then it is concatenated those vectors and passed through the final layer with sigmoid activation to obtain the probability of each of the 48 bits of the token codeword

```
x = tf.keras.layers.Reshape((48,))(x)
outputs = tf.keras.layers.Dense(48, activation='sigmoid')(x)# output layer
model = tf.keras.Model(inputs, outputs)
```

Results And Discussion

The model is trained for 30 epochs. It was generated 5000 independent samples for the validation dataset and another 5000 for the test dataset. The validation dataset was used to determine which epoch has the greatest accuracy for prediction of next codeword using the heuristic previously mentioned. The test dataset was used to obtain the results of this section. The number of distinct

tokens is 56916 therefore the probability of randomly choosing the correct codeword $1.756 \times 10^{-3}\%$ which serves as a baseline for the model. TensorFlow reports that the architecture contains 2,189,736 trainable parameters distributed across 23 dense layers.

To test the contribution of the context matrices in the prediction, it was also performed different types of ablation on the context vectors. More specifically it was performed substitution by zero ablation where the context vectors were substituted by zero, substitution by mean ablation where they were replaced by the mean of the context vectors in the test dataset itself and shuffle ablation where the context vectors were simply shuffled.

Table 1. Token prediction accuracy under different ablation settings for the deterministic model.

Method	**Top-1 Accuracy (%)**	**Top-10 Accuracy (%)**
No Ablation	22.08	29.92
Mean Ablation	20.06	27.38
Zero Ablation	19.34	26.72
Shuffle Ablation	18.54	25.88

Table 2. Token prediction accuracy under different ablation settings for the stochastic model.

Method	**Top-1 Accuracy (%)**	**Top-10 Accuracy (%)**
No Ablation	17.14	24.78
Mean Ablation	16.60	24.00
Zero Ablation	16.60	23.90
Shuffle Ablation	15.96	23.32

To assess the statistical significance of differences in accuracy, it was modeled prediction accuracy as a binomial proportion. The standard error (SE) for each accuracy p is estimated as

$$SE = \sqrt{\frac{p(1-p)}{N}} \tag{22}$$

where N is the number of samples.

For the comparison of two models, the standard error of the difference is approximated as

$$SE_{\text{diff}} = \sqrt{SE_1^2 + SE_2^2} \tag{23}$$

assuming independence between estimates. The corresponding z-score is then computed as

$$z = \frac{|p_1 - p_2|}{SE_{\text{diff}}} \tag{24}$$

As shown in Table 1 and Table 2, applying ablations to the input components corresponding to the context vectors results in a consistent drop in accuracy. For the deterministic model, the difference between the non-ablated setting and the substitution-by-mean ablation yields z-scores of 2.477 and 2.810 for top-1 and top-10 accuracy, respectively, indicating a statistically significant decrease in performance. These results suggest that the context vectors contain information relevant for prediction, as their perturbation leads to a measurable degradation in accuracy.

For the deterministic model, the difference between the non-ablated setting and shuffle ablation yields z-scores of 4.404 and 4.508 for top-1 and top-10 accuracy. This behavior is consistent with the interpretation that the context vectors encode information relevant to the current discourse context.

Under this view, shuffling the context vectors effectively replaces the original context with mismatched or unrelated information present on other context, which leads to incorrect predictions.

For the stochastic model, the difference between the non-ablated setting and shuffle ablation yields z-scores of 1.588 and 1.708 for top-1 and top-10 respectively, while the z-scores between the non-ablated setting and substitution-by-mean ablation are 0.721 and 0.908 for top-1 and top-10. These results indicate that, under the stochastic model, perturbations of the context vectors do not produce statistically significant changes in predictive performance under standard thresholds, suggesting a weaker and less consistent dependence on information encoded in the context vectors compared to the deterministic model.

For the deterministic model, the difference between the substitution-by-zero ablation and substitution-by-mean ablation yields z-scores of 0.905 and 0.743 for top-1 and top-10 accuracy, respectively, while for the stochastic model the corresponding values are 0.00 and 0.117. These results indicate that replacing the context vectors with their average value provides no statistically significant gain or loss in predictive performance for either model.

Conclusion and Future Work

This work proposes a novel technique for generating token's codewords where individual bits of the codeword are related to the semantics of the token as each bit is derived directly from the token's vector embeddings, where the interpretability of each bit enables better prediction by the model.

To encode the semantics of a sentence, the sequence of tokens would map to a discretized path that context matrices would follow in a higher dimensional sphere, since the Frobenius norm is kept constant by the already predefined update rule. The correlation of the context matrix with the underlying context comes from the fact that rotation matrix embeddings of each token are applied directly to the update rule and the ablation results provided supporting that the contribution of those matrices are statistically significant.

The work on this article has still room for improvement on many areas which are likely to increase accuracy and coherence of the language model and therefore should be studied more rigorously.

The choice of parameters for the non-linear system as well as the type of non-linear system that should encode the context must be examined more thoroughly as in this work it was only experimented empirically with some non-linear systems instead of a rigorous evaluation of the best choice of system.

When examining examples generated by the model, it was seen that many of the predicted tokens did not fit that position as it would be grammatically incorrect. Since the entire model is modular, one can adapt the heuristic of the scoring function to compute scores for only syntactically correct tokens during prediction without any change to the parameters of the neural network. The same modularity also allows the creation of a second classification model that can be used for re-ranking top-k predictions made with the heuristic.

It was also noticed that some of the sentences in training dataset would stop mid-way and some of the tokens were not grammatically valid as it was seen separations that shouldn't exist as for example *"ca n't"* instead of *"can't"*. Training the model on a dataset that is not affected by such issues can improve results as sentences are more predictable.

Also the current work gives a direction where improvement can happen for this sort of architecture as it was shown that through using codewords with individual bits tied to the semantics of the token can produce reasonable results, as semantically related bits are easier to predict than full vocabulary distributions. However, it is most likely the case that those token representations are

still sub-optimal for token prediction that can come from this line of research. Perhaps a different choice of principal components or a dimensionality reduction method such as autoencoders can yield better results.

Acknowledgement / Transparency

Parts of this manuscript were assisted by a language model for drafting and proofreading. All technical content, experiments, and results were generated and verified by the author.

Bibliography

[1] M. Honnibal and I. Montani, "spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing," 2017.

[2] T. G. Dietterich and G. Bakiri, "Solving multiclass learning problems via error-correcting output codes," Dec. 1994.

[3] S. Escalera, O. Pujol, and P. Radeva, "Loss-Weighted Decoding for Error-Correcting Output Coding.," 2008, pp. 117–122.

[4] Y. Oda, P. Arthur, G. Neubig, K. Yoshino, and S. Nakamura, "Neural Machine Translation via Binary Code Prediction." [Online]. Available: https://arxiv.org/abs/1704.06918

[5] R. Salakhutdinov and G. Hinton, "Semantic hashing," vol. 50, no. 7, pp. 969–978, July 2009.

[6] J. Tissier, C. Gravier, and A. Habrard, "Near-lossless Binarization of Word Embeddings," 2018, doi: 10.48550/ARXIV.1803.09065.

[7] A. Zhang, C. Su, Z. Wang, C. Zhou, and Y. Yang, "Reservoir computing-driven inverse dynamics for autonomous vehicle trajectory tracking control," *Sci. Rep.*, vol. 16, no. 1, p. 2559, Dec. 2025.

[8] F. Köster and A. Uchida, "Reservoir Computing as a Language Model." [Online]. Available: https://arxiv.org/abs/2507.15779

[9] V. Sharma and V. Raman, "Steering conceptual bias via transformer latent-subspace activation," June 2025.

[10] D. Nikolaev and S. Padó, "Investigating semantic subspaces of Transformer sentence embeddings through linear structural probing," Oct. 2023.

[11] Y. Sakemi, K. Morino, T. Leleu, and K. Aihara, "Model-size reduction for reservoir computing by concatenating internal states through time," *Sci. Rep.*, vol. 10, no. 1, p. 21794, Dec. 2020.

[12] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," Jan. 2019.

[13] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with Rotary Position Embedding," Apr. 2021.

[14] J. Mu, S. Bhat, and P. Viswanath, "All-but-the-Top: Simple and Effective Postprocessing for Word Representations," *CoRR*, 2017, [Online]. Available: http://arxiv.org/abs/1702.01417

[15] S. Lahiri, "Complexity of Word Collocation Networks: A Preliminary Structural Analysis," in *Proceedings of the Student Research Workshop at the 14th Conference of the European Chapter of the Association for Computational Linguistics*, Gothenburg, Sweden: Association for Computational Linguistics, Apr. 2014, pp. 96–105. [Online]. Available: http://www.aclweb.org/anthology/E14-3011

[16] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” June 2016.

[17] M. Lee, “Mathematical analysis and performance evaluation of the GELU activation function in deep learning,” *J. Math.*, vol. 2023, pp. 1–13, Aug. 2023.

[18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[19] K. Cho *et al.*, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” June 2014.